Lab 3: Architectural Pattern Selection & Justification

Second-Hand Product Escrow Marketplace

Name: Akhila

SRN: PES2UG24CS043

Architecture Selection

We chose Layered Architecture for the Second-Hand Product Escrow Marketplace System.

Why this architecture makes sense here

The whole point of this system is trust — a buyer's money has to sit safely in escrow while an independent inspector checks the product, and only then should the seller get paid. That kind of step-by-step, rule-based flow (order → payment hold → inspection → approval → release) maps really naturally onto layers, where each layer only talks to the one next to it and nothing jumps ahead in the process.

- **Reason 1 — Clear separation between UI, business rules, and data:** Buyers and sellers interact only through the Presentation layer (the UI component), while all the actual decision-making — locking funds, checking inspection status, approving release — lives in the Business layer. This means we can redesign the buyer-facing app later (say, a mobile app) without touching how escrow or payment logic works underneath.
- **Reason 2 — Predictable, sequential transaction flow:** Escrow marketplaces don't need every component scaling independently or reacting to unpredictable traffic spikes the way, say, a social feed would. The transaction always follows the same order (Order → Payment → Inspection → Escrow release), so a layered structure that enforces this sequencing top-to-bottom fits better than the added complexity of microservices, and it's simpler to build and test than a full client-server split.

Security Advantage

Because the Business layer sits between the UI and the Data layer, no request from a buyer or seller can touch the database directly — every action (like approving a product or releasing funds) has to pass through the Escrow Ledger and Order Manager components first. This gives us one clear checkpoint where we can enforce rules like "funds can only be released after both an inspection report AND buyer approval exist," instead of relying on the UI to behave correctly.

Performance Benefit

Since the Payment Service and Inspection Service are separated out as their own components within the Business layer, a slow response from the payment gateway or a delay in inspection reporting doesn't block other unrelated actions — such as a buyer simply browsing listings through the UI component. Each component only does its one job, so the system isn't forced to wait on one slow external service before doing anything else.